DASAR PEMROGRAMAN PERANGKAT BERGERAK (DPPB) | MODUL 1 - 4 | DART

LATIHAN PEMROGRAMAN DART

Latihan Soal Berbasis *Cheat Sheet Ujian Praktikum* — Fondasi Dart untuk Flutter

Nama	.....	**Kelas**	.....
Tanggal	..... / ..... /	**Nilai**	..... / 100

Petunjuk Pengerjaan

1. Kerjakan soal secara berurutan dari Bagian A sampai Bagian D. Waktu pengerjaan yang disarankan: **90 menit**.
2. **Bagian A & B**: pilih / tulis satu jawaban yang paling tepat pada lembar jawaban.
3. **Bagian C**: lengkapi kode pada titik-titik (`.....`) atau jawab secara singkat dengan bahasa sendiri.
4. **Bagian D**: tulis program lengkap lalu **uji jalankan** di `dartpad.dev`, VS Code (*Run*), atau terminal dengan `dart nama_file.dart`.
5. Kunci jawaban Bagian A, B, dan C ada di halaman terakhir. Kerjakan semua soal terlebih dahulu sebelum memeriksanya.
6. Gunakan **tanpa membuka cheat sheet** terlebih dahulu (simulasi ujian), baru gunakan cheat sheet untuk remedial.

Bobot Penilaian

Bagian	Bentuk Soal	Bobot
Bagian A	Pilihan ganda (16 soal)	25%
Bagian B	Analisis / prediksi output (9 soal)	25%
Bagian C	Lengkapi kode & esai singkat (6 soal)	15%
Bagian D	Praktik / live coding (8 soal)	35%

Cakupan Materi (sesuai cheat sheet)

1. Struktur program & menjalankan
2. Variabel & tipe data
3. Null safety
4. Operator
5. String & interpolasi
6. Input keyboard
7. Percabangan
8. Perulangan
9. Fungsi
10. List
11. Set
12. Map
13. Queue
14. Exception handling
15. Class & objek
16. Constructor (4 bentuk)
17. Method, getter & setter
18. Enkapsulasi
19. Inheritance & @override
20. Polimorfisme
21. Agregasi & komposisi
22. Pola menu live coding
23. Jebakan umum

Catatan: cheat sheet DPPB yang dijadikan rujukan memuat materi **Dart** secara utuh (Modul 1–4). Dart adalah bahasa dasar yang dipakai Flutter, sehingga seluruh isi latihan ini menjadi bekal langsung untuk memrogram widget Flutter pada materi berikutnya (class StatelessWidget, StatefulWidget, ListView, dsb.).

BAGIAN A — PILIHAN GANDA • 16 soal • pilih SATU jawaban paling tepat

A1. Pernyataan yang benar mengenai perbedaan `final` dan `const` adalah ...

a. `const` boleh diisi dari hasil input runtime, sedangkan `final` tidak

b. `final` boleh diisi dari runtime (sekali isi), sedangkan `const` wajib nilai compile-time

c. Keduanya sama-sama boleh diubah nilainya kapan saja

d. `const` hanya bisa digunakan untuk tipe `String`

A2. Perhatikan cuplikan kode berikut:

```
String? nama;
print(nama ?? 'Anonim');
nama ??= 'Budi';
print(nama);
```

a. null

b. Anonim

c. Anonim, kemudian Budi

d. Budi

A3. Urutan hasil ketiga `print` pada kode di bawah adalah ...

```
print(7 ~/ 2);
print(7 % 2);
print(7 / 2);
```

a. 3.5 / 1 / 3

b. 3 / 1 / 3.5

c. 3 / 0 / 3.5

d. 2 / 1 / 3.5

A4. Output dari kode berikut adalah ...

```
String teks = 'Dart Lang';
print(teks.substring(0, 4).toUpperCase());
print(teks.split(' ').length);
```

a. DART dan 9

b. DART dan 2

c. Dart dan 2

d. DART LANG dan 2

A5. Manakah pernyataan yang benar tentang `List` dan `Set`?

a. Set boleh menyimpan duplikat, sedangkan List tidak

b. List menyimpan data terurut dan boleh duplikat, sedangkan Set hanya menyimpan data unik

c. Keduanya menyimpan data berpasangan key → value

d. Keduanya wajib dideklarasikan kosong terlebih dahulu

A6. Perhatikan deklarasi `Set<String> hadir = ...;` (set kosong). Manakah cara deklarasi yang benar?

a. `Set<String> hadir = {};`

b. `Set<String> hadir = <String>{};`

c. `Set<String> hadir = [];`

d. `var hadir = {};`

A7. Output dari kode berikut adalah ...

```
Map<String, String> kamus = {'TULT': 'Telkom Tower'};
kamus.putIfAbsent('GKU', () => 'Gedung Kuliah Umum');
print(kamus.length);
print(kamus.containsKey('GKU'));
```

a. 1, lalu false

b. 2, lalu false

c. 2, lalu true

d. 3, lalu true

A8. Output dari kode berikut adalah ...

```
import 'dart:collection';
void main() {
  var antre = Queue<String>();
  antre.add('Andi');
  antre.add('Budi');
  antre.addFirst('Citra');
  print(antre.removeFirst());
  print(antre.first);
}
```

a. Citra, lalu Andi

b. Andi, lalu Citra

c. Budi, lalu Andi

d. Citra, lalu Budi

A9. Hasil cetak dari `print(int.tryParse('12a') ?? 0);` adalah ...

a. 12

b. 0

c. null

d. FormatException (program error)

A10. Perhatikan blok `try – on – catch – finally`. Pernyataan yang benar tentang blok `finally` adalah ...

a. dijalankan hanya jika tidak terjadi error

b. dijalankan hanya jika terjadi error

c. selalu dijalankan, baik terjadi error maupun tidak

d. berfungsi untuk melempar error baru

A11. Output dari program berikut adalah ...

```
class Pasien {
  int _umur;
  Pasien(this._umur);
  int get bacaUmur => _umur;
  set ubahUmur(int nilai) {
    if (nilai >= 0) { _umur = nilai; }
    else { print('Error: umur negatif'); _umur = 0; }
  }
}
void main() {
  var p = Pasien(20);
  p.ubahUmur = -5;
  print(p.bacaUmur);
}
```

a. 20

b. -5

c. Error: umur negatif, lalu 0

d. Error: umur negatif, lalu 20

A12. Perhatikan potongan berikut. Jika dijalankan, `p.bekerja();` akan menampilkan ...

```
PegawaiRS p = Dokter('B', 'D-2', 'Anak');   // PegawaiRS = induk
p.bekerja();                                 // Dokter punya @override bekerja()
```

a. versi milik induk (PegawaiRS)

b. versi milik anak (Dokter) karena method sudah di-`@override`

c. error, karena tipe variabelnya induk

d. tidak ada output sama sekali

A13. Class `KamarInap` membuat `LogRekamMedis` sendiri **di dalam** constructor miliknya. Hubungan tersebut disebut ...

a. agregasi

b. komposisi

c. pewarisan (inheritance)

d. polimorfisme

A14. **Bentuk constructor dan kelebihan dari kode berikut adalah ...**

```
Warga({required this.nik, required this.nama});
var w = Warga(nama: 'Asep', nik: '327');
```

a. positional parameter — urutan pengisian wajib urut

b. named parameter — urutan pengisian bebas sehingga tidak tertukar

c. named constructor — hanya dipakai sekali saja

d. initializer — memberi nilai default pada properti

A15. **Arti penggunaan `super.nama` dan `super.idPegawai` pada constructor anak adalah ...**

```
class Dokter extends PegawaiRS {
  Dokter(super.nama, super.idPegawai, this.spesialisasi);
}
```

a. mendeklarasikan properti baru milik anak

b. meneruskan nilai ke constructor milik class induk

c. membuat objek induk baru secara otomatis

d. memanggil method induk bernama super

A16. **Output dari kode berikut adalah ...**

```
var n = [75, 90, 80];
n.sort((a, b) => b.compareTo(a));
print(n);
```

a. [75, 80, 90]

b. [90, 80, 75]

c. [75, 90, 80]

d. error, List tidak bisa diurutkan

BAGIAN B — ANALISIS OUTPUT • 9 soal • tuliskan output persis seperti keluaran program

B1. Tuliskan output program berikut:

```
void main() {
  String teks = ' Dart ';
  print(teks.trim().toUpperCase());
  print(teks.length);
}
```

B2. Tuliskan output program berikut:

```
void main() {
  int total = 0;
  for (int i = 1; i <= 5; i++) {
    if (i % 2 == 0) continue;
    total += i;
  }
  print(total);
}
```

B3. Tuliskan output program berikut:

```
void main() {
  var angka = [3, 1, 2, 1];
  angka.add(4);
  angka.sort();
  angka.removeAt(0);
  print(angka);
  print(angka.length);
}
```

B4. Tuliskan output program berikut:

```
void main() {
  Map<String, int> nilai = {'Andi': 80, 'Budi': 90};
  nilai['Citra'] = 70;
  nilai['Budi'] = 95;
  print(nilai.length);
  print(nilai['Budi']);
  print(nilai.containsKey('Dedi'));
}
```

B5. Tuliskan output program berikut:

```
void main() {
  var a = {1, 2, 3};
  var b = {2, 3, 4};
  print(a.union(b));
  print(a.intersection(b));
}
```

B6. Tuliskan output program berikut:

```
void main() {
  try {
    int n = int.parse('dua');
    print(n);
  } on FormatException {
    print('Input harus angka!');
  } finally {
    print('Selesai dicek');
  }
}
```

B7. Tuliskan output program berikut:

```
class A {
  void halo() => print('Halo A');
}
class B extends A {
  @override
  void halo() {
    super.halo();
    print('Halo B');
  }
}
void main() {
  A x = B();
  x.halo();
}
```

B8. Tuliskan output program berikut:

```
class Tabungan {
  double _saldo = 0;
  set setor(double n) {
    if (n > 0) { _saldo += n; }
    else { print('Nominal salah'); }
  }
  double get lihat => _saldo;
}
void main() {
  var t = Tabungan();
  t.setor = 100000;
  t.setor = -5000;
  print(t.lihat);
}
```

B9. Tuliskan output program berikut:

```
import 'dart:collection';
void main() {
  var antre = Queue<String>();
  antre.add('Andi');
  antre.add('Budi');
  antre.addFirst('Citra');
  print(antre.removeFirst());
  print(antre.first);
}
```

BAGIAN C — LENGKAPI KODE & ESAI SINGKAT • 6 soal

C1. (Esai) Jelaskan fungsi masing-masing operator null safety di bawah ini, lalu berikan satu contoh pemakaian untuk setiap operator.

a. `?` b. `??` c. `??=` d. `!` e. `?.`

Jawab:

```
.....
.....
.....
```

C2. Lengkapi fungsi-fungsi berikut pada titik-titik (`.....`) agar sesuai ketentuan dalam komentar.

```
int tambah(int a, int b) => .....;              // hasilkan a + b
double ongkir(double berat) => .....;           // berat * 10000

void cetakProfil({required String nama, int umur = 0}) {
  print('.....');                               // contoh: Andi (20 tahun)
}
void beriTag(String pesan, [String tag = 'INFO']) {
  print('.....');                               // contoh: [INFO] Siap
}
void main() {
  print(tambah(2, 3));
  print(ongkir(2.5));
  cetakProfil(nama: 'Andi', umur: 20);
  beriTag('Siap');
}
```

C3. Tulislah **empat bentuk constructor** untuk class `Warga` berikut (posisional, named parameter, named constructor, initializer) beserta cara memanggilnya di `main()`.

```
class Warga {
  String nik;
  String nama;
  // 1) posisional
  // 2) named parameter {required ...}
  // 3) named constructor Warga.anonym()
  // 4) initializer  (nik diisi default '327301')
}
void main() {
  // tulis contoh pemanggilan ke-empatnya
}
```

C4. Lengkapi pola **input angka aman** + **total & rata-rata** berikut. Pastikan program tidak crash bila input bukan angka.

```
import 'dart:io';
void main() {
  stdout.write('Jumlah data: ');
  int? n = ..........;                        // tryParse + ?? '' , hasil null bila gagal
  if (n == null) {
    print('Input harus angka!');
  } else {
    List<double> daftar = [];
    for (int i = 0; i < n; i++) {
      stdout.write('Data ke-${i + 1}: ');
      double? v = ..........;                 // tryParse, hasil null bila gagal
      if (v != null) daftar.add(v);
    }
    double total = 0;
    for (var x in daftar) { total += x; }
    print('Total    : Rp${..........}');    // tanpa digit desimal
    print('Rata-rata: ${..........}');      // 2 digit desimal
  }
}
```

C5. Lengkapi class `Pasien` berikut agar menerapkan **enkapsulasi** (atribut privat + getter + setter dengan validasi).

```
class Pasien {
  String nama;
  int ..........;                            // atribut privat (tanda _)
  Pasien(this.nama, this..........);

  int get bacaUmur => ..........;           // getter: baca data

  set ubahUmur(int nilai) {                 // setter: ubah data + validasi
    if (nilai >= 0) { ..........; }
    else { print('Error: umur negatif'); ..........; }
  }
}
void main() {
  var p = Pasien('Asep', 0);
  p.ubahUmur = 45;
  print(p.bacaUmur);
  p.ubahUmur = -3;                          // coba bandingkan output-nya
}
```

C6. **(Esai) Jawablah dengan bahasamu sendiri:**

a. Apa itu **polimorfisme**? Sebutkan dua syarat agar polimorfisme bisa terjadi.

b. Apa yang terjadi bila `PegawaiRS p = Dokter('A', 'D-1', 'Gigi');` lalu `p.bekerja();` dijalankan? Mengapa demikian?

c. Berbeda dengan apa yang dimaksud **abstraksi** (`class Bentuk { ... }` lalu `implements`)?

Jawab:

.....
.....
.....
.....

TIPS & JEBAKAN UMUM SEBELUM MASUK BAGIAN D • cek cepat ala cheat sheet

- Setiap statement diakhiri `;`, kurung `{ }` seimbang, nama class diawali **Kapital**.
- `import 'dart:io'` untuk `stdin/stdout` → `import 'dart:collection'` untuk `Queue`.
- `int.parse` bisa melempar `FormatException` → pakai `int.tryParse(...) ?? 0` untuk input aman.
- Akses `list[99]` bisa `RangeError` → cek `list.length` terlebih dahulu.
- Atribut `_private` tidak bisa diakses dari `main()` → sediakan getter/setter.
- Constructor anak wajib `super(...)` dan jangan lupa `@override`.
- Tambah objek ke `List`, cetak memakai **for-in** (bukan `print` manual satu-satu).
- Variabel `String?` wajib diberi `?? default` sebelum dicetak/dipakai.
- Interpolasi ekspresi pakai `${...}`, variabel cukup `$x`.
- Urutan aman live coding: **1)** class + property → **2)** constructor → **3)** method (validasi/logika) → **4)** `main()`: buat objek → masukkan `List` → cetak pakai for-in.

BAGIAN D — PRAKTIK / LIVE CODING • 8 soal • buat file .dart terpisah untuk tiap soal lalu jalankan

D1. Sapa Pengguna

dart:io • input • interpolasi

Buat program `sapa.dart` yang:

- Menampilkan prompt tanpa enter memakai `stdout.write()`, lalu membaca **Nama**, **Umur** (int), dan **Berat** (double) menggunakan `stdin.readLineSync()`.
- Membaca data dengan aman: `readLineSync()` bertipe `String?`, selalu beri `?? ''` / `?? '0'` agar tidak error saat Enter kosong.
- Mencetak: `Halo <nama>, umur <umur> tahun, berat <berat> kg.`

```
Nama: Dewi
Umur: 19
Berat: 55.5
Halo Dewi, umur 19 tahun, berat 55.5 kg
```

✓ **Poin penilaian:** pemakaian stdout.write & readLineSync + ?? default (3) • konversi tipe int/double (3) • interpolasi & output sesuai (4)

D2. Sensor Udara

class • named constructor • List • for-in

Buat program `sensor.dart` dengan ketentuan:

- Class `Sensor` berisi properti `lokasi` (String), `aqi` (int), `status` (String default `'Belum'`).
- Constructor **named parameter**: `Sensor({required this.lokasi, required this.aqi});`
- Method `analisis()` mengisi `status`: AQI ≤ 50 → `'Bersih'`, selain itu → `'Polusi'` (pakai ternary).
- Method `cetak()` menampilkan `lokasi | AQI: aqi | status`.
- Di `main()`: buat **minimal 3 objek**, simpan ke `List<Sensor>`, lalu telusuri dengan **for-in** sambil memanggil `analisis()` dan `cetak()`.

```
Dago | AQI: 45 | Bersih
Alun-Alun | AQI: 80 | Polusi
Cihampelas | AQI: 30 | Bersih
```

✓ **Poin penilaian:** class + property (2) • named parameter (2) • method analisis & cetak (3) • List + for-in + output (3)

D3. Rekap Nilai Mata Kuliah `input aman • percabangan • total & rata-rata`

Buat program `rekap.dart` dengan ketentuan:

- Minta jumlah mahasiswa memakai `int.tryParse(stdin.readLineSync() ?? '')`. Bila `null`, tampilkan `'Input harus angka!'` dan program berhenti (tidak crash).
- Baca nilai tiap mahasiswa (0–100) ke dalam `List<double>`; nilai yang gagal dibaca cukup dilewati.
- Hitung **total** dan **rata-rata** dengan perulangan (tanpa method khusus), cetak dengan `toStringAsFixed`: total 0 desimal, rata-rata 2 desimal.
- Tentukan kategori nilai tertinggi memakai `if – else if – else`: ≥ 85 = A, ≥ 75 = B, selain itu C.
- Tampilkan juga jumlah data yang berhasil dibaca (pakai `isEmpty` bila tidak ada data sama sekali).

```
Jumlah data: 3
Data ke-1: 90
Data ke-2: 78
Data ke-3: 65
Total    : Rp233
Rata-rata: 77.67
Kategori : B
```

✓ **Poin penilaian:** input aman tryParse + cek null (3) • List + perulangan total/rata (3) • if-else if-else (2) • format angka & output (2)

D4. Antrean Klinik `Queue • menu while(true)`

Buat program `antrean.dart` dengan ketentuan:

- Gunakan `import 'dart:collection';` dan `Queue<String>` untuk menyimpan antrean pasien.
- Menu berjalan terus memakai `while (true)`:

```
1. Tambah antrean
2. Layani pasien
3. Lihat antrean
4. Keluar
```

- Pilihan **1**: tanya nama pasien → `add()` (masuk belakang). Pilihan **2**: `removeFirst()` (layani paling depan) — bila antrean kosong tampilkan `'Antrean kosong!'`.
- Pilihan **3**: tampilkan urutan antrean beserta jumlahnya (`length`, `isEmpty`).
- Pilihan **4**: `break` keluar dari perulangan. Pilihan lain: `'Menu tidak tersedia!'`.

```
Pilih menu (1-4): 1   Nama: Andi
Pilih menu (1-4): 1   Nama: Budi
Pilih menu (1-4): 2   Andi dilayani. Sisa antrean: 1
Pilih menu (1-4): 3   1. Budi
Pilih menu (1-4): 4   Keluar...
```

✓ **Poin penilaian:** Queue + import benar (3) • menu while(true) + break (3) • penanganan antrean kosong (2) • output sesuai (2)

D5. Rekening Bank `enkapsulasi • getter/setter • validasi`

Buat program `rekening.dart` dengan ketentuan:

- Class `Rekening` dengan properti `pemilik` (String publik) dan `_saldo` (**private**, double).
- Method `setor(double n)`: hanya menambah bila `n > 0`, selain itu cetak `'Nominal harus lebih dari 0!'`.
- Method `tarik(double n)`: hanya mengurangi bila `n > 0` dan `n <= _saldo`, selain itu cetak `'Saldo tidak mencukupi!'`.
- Gunakan **getter** `saldo` untuk membaca nilai privat dari `main()` (dilarang mengakses `_saldo` langsung).
- Override method `toString()` agar `print(rek)` rapi: `Pemilik: Andi | Saldo: Rp150000`.
- Di `main()`: uji setor, tarik melebihi saldo, dan tarik negatif.

```
Saldo awal : Rp0
Setor 200000 -> Rp200000
Tarik  50000 -> Rp150000
Tarik 999999 -> Saldo tidak mencukupi!
Setor   -1000 -> Nominal harus lebih dari 0!
Pemilik: Andi | Saldo: Rp150000
```

✓ **Poin penilaian:** atribut private (2) • method + validasi (4) • getter dari main (2) • toString + output (2)

D6. Data Pegawai Rumah Sakit `extends • @override • polimorfisme • getter`

Buat program `pegawai.dart` dengan ketentuan:

- Class induk `PegawaiRS`: properti `nama`, `idPegawai`; method `info()` dan `bekerja()`; constructor positional dengan `super` untuk anak.
- Dua class anak: `Dokter` (tambahan `spesialisasi`) dan `Perawat` (tambahan `shift`), keduanya **meng-`@override`** method `bekerja()` dengan output berbeda.
- Setiap anak memiliki **getter** `gaji` (mis. Dokter Rp12.000.000, Perawat Rp7.000.000).
- Di `main()`: buat `List<PegawaiRS>` berisi minimal 3 objek campuran, lalu:
  - telusuri dengan for-in untuk memanggil `bekerja()` (bukti polimorfisme),
  - hitung **total gaji** semua pegawai,
  - cari 1 pegawai berdasarkan ID dengan `firstWhere` yang dibungkus `try – on Error` (atau cek `isEmpty` dulu) agar tidak error bila tidak ditemukan.

```
Dr. Andi - D-001 | Penyakit Dalam
Suster Sari - P-002 | Pagi
Total gaji: Rp31000000
Cari ID D-003 -> Tidak ditemukan
```

✓ **Poin penilaian:** class induk + anak + super (3) • @override (2) • List + for-in polimorfisme (2) • total getter + pencarian aman (3)

D7. Agen Ekspedisi (Jurus Live Coding) menu • class induk • override • List • total

Buat program `ekspedisi.dart` — soal utama yang menggabungkan seluruh materi:

- Class induk `Paket`: properti `namaPengirim`, `tujuan`, `beratKg`, dan method **abstrak/bermethod body** `hitungOngkir()`.
- Dua anak: `PaketReguler` (ongkir = berat × 7000) dan `PaketKilat` (ongkir = berat × 15000), keduanya `@override hitungOngkir()`.
- Class `AgenEkspedisi`: `List<Paket> daftarPaket`, `double _totalPendapatan` (private) + **getter** `totalPendapatan`, method `terimaPaket(Paket p)` yang menyimpan ke List dan menambahkan hasil `p.hitungOngkir()` (memakai referensi induk → polimorfisme).
- Menu `while (true)`: **1** Terima paket (tanya jenis 1/2, pengirim, tujuan, berat — pakai `double.tryParse` + cek null), **2** Daftar paket (for-in), **3** Total pendapatan (getter + `toStringAsFixed(0)`), **4** Keluar (`break`), selain itu `'Menu tidak tersedia!'`.

```
1. Terima 2. Lihat 3. Total 4. Keluar
Pilih menu (1-4): 1   Jenis (1=Reguler/2=Kilat): 2 ... Berat: 1.5
Paket kilat tujuan Bandung diterima agen!
1. Terima 2. Lihat 3. Total 4. Keluar
Pilih menu (1-4): 3   Total pendapatan: Rp27000
Pilih menu (1-4): 4   Keluar...
```

✓ **Poin penilaian:** class induk + 2 anak + override (4) • List + getter total + private (3) • menu while + break (2) • input aman + output (1)

D8. Agregasi vs Komposisi asosiasi

Buat program `asosiasi.dart` yang membuktikan perbedaan kedua hubungan:

- **AGREGASI (longgar)**: class `Klinik` punya `List<Dokter>` sebagai wadah. Objek `Dokter` dibuat di **luar** class Klinik (di `main()`) lalu disuntikkan lewat method `tambahDokter(Dokter d)`.
- **KOMPOSISI (mutlak)**: class `KamarInap` memiliki `logHarian` bertipe `LogRekamMedis` yang **dibuat sendiri di dalam** constructor-nya (pakai `late`).
- Di `main()`: buat objek Dokter, masukkan ke Klinik, hapus dari List (simulasi induk hilang) lalu cetak bahwa objek Dokter **masih hidup** dan tetap bisa dipanggil. Bandingkan dengan log pada KamarInap yang hanya hidup selama KamarInap ada.
- Cetak penjelasan singkat hubungan yang dipakai pada tiap class (sesuai template kesimpulan cheat sheet).

```
Klinik menampung: Dr. Andi
Klinik berhenti menampung: Dr. Andi
Dr. Andi masih hidup (agregasi - objek dibuat di luar)
Kamar 301 -> log harian dibuat di dalam constructor (komposisi)
```

✓ **Poin penilaian:** class Klinik + List + method suntik (3) • class KamarInap + pembuatan objek di dalam (3) • percobaan di main & kesimpulan (4)

KUNCI JAWABAN • Bagian A, B, dan C (Bagian D dinilai berdasarkan rubrik)

Bagian A — Pilihan Ganda

A1. b — `final` boleh berasal dari runtime (sekali isi), `const` wajib nilai compile-time.

A2. c — `nama` awal null → `??` mencetak *Anonim*; `??=` mengisi karena masih null → cetakan kedua *Budi*.

A3. b — `~/` bagi bulat (3), `%` sisa bagi (1), `/` bagi biasa (3.5).

A4. b — `substring(0,4)` = 'Dart' → 'DART'; `split(' ')` menghasilkan 2 elemen.

A5. b — List terurut & boleh duplikat; Set unik (tidak boleh duplikat).

A6. b — `{}` tanpa tipe akan dibaca sebagai Map, sehingga Set kosong wajib `<String>{}`.

A7. c — `putIfAbsent` menambah key baru (length 2) dan `containsKey('GKU')` menjadi true.

A8. a — `addFirst` menyisipkan 'Citra' di depan, `removeFirst()` mengeluarkannya lebih dulu (FIFO), sisanya tetap 'Andi'.

A9. b — `tryParse` menghasilkan null bila gagal, lalu `?? 0` menggantinya dengan 0 (tidak error).

A10. c — blok `finally` selalu dijalankan, baik terjadi error maupun tidak.

A11. c — setter menolak nilai negatif: mencetak pesan error lalu `_umur` di-set 0.

A12. b — dengan referensi induk `PegawaiRS` tetapi objek `Dokter`, method yang jalan adalah versi anak karena `@override` (polimorfisme).

A13. b — objek anak dibuat di DALAM induk → komposisi (ikut hilang bersama induk).

A14. b — memakai named parameter `{required ...}` sehingga urutan pengisian data bebas dan tidak tertukar.

A15. b — `super.nama` meneruskan argumen ke constructor class induk `PegawaiRS`.

A16. b — `b.compareTo(a)` membalik urutan menjadi menurun: [90, 80, 75].

Bagian B — Analisis Output

B1.

```
DART
7
```

(trim membuang spasi tepi → 'DART'; panjang ' Dart ' = 7 karakter)

B2.

```
9
```

(`continue` melewatkan bilangan genap, yang dijumlahkan 1+3+5 = 9)

B3.

```
[1, 2, 3, 4]
4
```

(setelah sort = [1,1,2,3,4], `removeAt(0)` membuang elemen pertama)

B4.

```
3
95
false
```

(key 'Citra' ditambahkan, 'Budi' di-update, 'Dedi' tidak ada)

B5.

```
{1, 2, 3, 4}
{2, 3}
```

(union = gabungan tanpa duplikat, intersection = irisan)

B6.

```
Input harus angka!
Selesai dicek
```

(`int.parse('dua')` melempar FormatException → ditangkap blok on, lalu `finally` tetap jalan)

B7.

```
Halo A
Halo B
```

(objek `B` dengan referensi induk `A`: method override dijalankan, `super.halo()` memanggil versi induk lebih dulu)

B8.

```
Nominal salah
100000.0
```

(setter menolak -5000, saldo bertambah hanya saat setor 100000)

B9.

```
Citra
Andi
```

(`removeFirst()` melayani paling depan, sisanya 'Andi', 'Budi')

Bagian C — Contoh Jawaban

C1.

- `String? nama;` → tipe **nullable**, boleh berisi null.
- `??` → memberi nilai pengganti bila null. Contoh: `print(nama ?? 'Anonim');`
- `??=` → mengisi hanya jika masih null. Contoh: `nama ??= 'Budi';`
- `!` → memaksa nilai non-null (bisa error/null-check exception bila null). Contoh: `print(nama!.length);`
- `?.` → akses aman bila null (hasilnya null, tidak crash). Contoh: `print(nama?.length);`

C2.

```
int tambah(int a, int b) => a + b;
double ongkir(double berat) => berat * 10000;

void cetakProfil({required String nama, int umur = 0}) {
  print('$nama ($umur tahun)');
}
void beriTag(String pesan, [String tag = 'INFO']) {
  print('[$tag] $pesan');
}
// output: 5 / 25000.0 / Andi (20 tahun) / [INFO] Siap
```

C3.

```
class Warga {
  String nik;
  String nama;
  // 1) posisional
  Warga(this.nik, this.nama);
  // 2) named parameter
  Warga({required this.nik, required this.nama});
  // 3) named constructor
  Warga.anonym() { nik = '-'; nama = '-'; }
  // 4) initializer
  Warga(this.nama) : nik = '327301';
}
void main() {
  print(Warga('327', 'Asep'));                        // 1
  print(Warga(nama: 'Budi', nik: '328'));             // 2 (urutan bebas)
  print(Warga.anonym());                             // 3
  print(Warga('Caca'));                              // 4
}
```

C4.

```
int? n = int.tryParse(stdin.readLineSync() ?? '');
...
double? v = double.tryParse(stdin.readLineSync() ?? '');
...
print('Total    : Rp${total.toStringAsFixed(0)}');
print('Rata-rata: ${(total / daftar.length).toStringAsFixed(2)}');
```

Kunci utama: `tryParse` → hasil `null` bila input bukan angka, sehingga program tidak crash; `toStringAsFixed(n)` untuk membatasi digit desimal.

C5.

```
class Pasien {
  String nama;
  int _umur;                          // atribut privat
  Pasien(this.nama, this._umur);

  int get bacaUmur => _umur;          // getter

  set ubahUmur(int nilai) {
    if (nilai >= 0) { _umur = nilai; }
    else { print('Error: umur negatif'); _umur = 0; }
  }
}
// output: 45, lalu Error: umur negatif → 0
```

C6.

- **(a)** Polimorfisme adalah kemampuan satu nama method yang sama menghasilkan perilaku/output berbeda pada tiap class anak. Syaratnya: **1)** method anak menggunakan `@override`, **2)** ada **referensi induk** yang menampung objek anak (mis. `PegawaiRS p = Dokter(...);`).
- **(b)** Yang dijalankan adalah implementasi milik `Dokter`, karena pada runtime objek bertipe `Dokter`; Dart memanggil versi paling akhir yang di-`@override`.
- **(c)** Abstraksi menekankan **"apa yang harus dikerjakan"** tanpa isi: method didefinisikan tanpa badan lalu diwajibkan oleh class lain memakai `implements`/`extends` (mis. `class Bentuk { void gambar(); }`), sedangkan polimorfisme adalah **pelaksanaan berbeda** dari method yang sama pada class anak.

Bagian D (Praktik): tidak ada kunci jawaban tunggal — nilai diberikan berdasarkan poin penilaian pada tiap soal. Kriteria umum: program **berjalan tanpa error**, konsep yang diminta benar dipakai (class, List, Queue, getter/setter, `@override`, dst.), penanganan input aman, dan output sesuai contoh.